

Artificial Intelligence

UNIT-III

Knowledge representation issues, predicate logic- logic programming, semantic nets- frames and inheritance, constraint propagation, representing knowledge using rules, rules-based deduction systems.

Reasoning under uncertainty, review of probability, Bayes' probabilistic interferences and Dempster-Shafer theory.

Knowledge Representation

- Knowledge representation (KR) is a fundamental area of artificial intelligence that focuses on how to formally represent information about the world in a form that a computer system can utilize to solve complex tasks.
- Knowledge representation (KR) is an important issue in both cognitive science and artificial intelligence.
 - In cognitive science, it is concerned with the way people store and process information and
 - In artificial intelligence (AI), main focus is to store knowledge so that programs can process it and achieve human intelligence.

Representation of Knowledge in AI

Knowledge representation (KR) is about **how information and facts about the world are stored inside an AI system** so that it can reason, solve problems, and make decisions.

Think of it like how we humans use different forms of representation:

- Words (language)
- Diagrams

- Symbols
- Numbers
- AI systems also need their own **representations of knowledge**.

Knowledge Representation Issues

When representing knowledge, AI faces several **key issues**:

1. **Representational Adequacy**
 - Can the system represent all the knowledge it needs?
 - Example: Representing facts like *"Cats are mammals"* or rules like *"If it rains, the ground gets wet."*
2. **Inferential Adequacy**
 - Can the system **draw new conclusions** from the stored knowledge?
 - Example: If the system knows *"All mammals are warm-blooded"* and *"Cats are mammals"*, it should infer *"Cats are warm-blooded."*
3. **Inferential Efficiency**
 - Can the system **find answers quickly** without wasting time or resources?
 - Example: A search engine should find relevant results in seconds, not hours.
4. **Acquisitional Efficiency**
 - Can the system **easily acquire new knowledge** and update old information?
 - Example: Adding new medical facts into a diagnostic AI system without rebuilding it from scratch.

5. **Trade-off Between Expressiveness and Efficiency**

- More expressive knowledge (like natural language) may be harder to process.
- Less expressive forms (like numbers) are faster but may miss details.
- AI has to balance between **richness of knowledge** and **speed of reasoning**.

Why KR is Important in AI

1. **Storing Information** – Computers need a systematic way to keep knowledge.
2. **Reasoning** – Once stored, systems should make **inferences** (draw conclusions).
3. **Problem Solving** – Used in expert systems, planning, and natural language processing.
4. **Communication** – Allows humans to interact with AI in understandable terms.

A **fact** in logic or knowledge representation is simply:

A statement that is true about the world.

1. In everyday meaning:

- A fact = something that is objectively true.

- Example:
 - “ The sun rises in the east.”
 - “ Water boils at 100° C (at sea level).”

2. In AI / Predicate Logic:

- A **fact** is a basic piece of knowledge stored in the knowledge base.
- It usually has **no conditions**— just a truth.

Example in **propositional logic**:

- PP: “ It is raining.” (if true, it’ s a fact)

Example in **predicate logic**:

- $\text{Man}(\text{Socrates})$ $\text{Man}(\text{Socrates})$ = Socrates is a man.
- $\text{Mortal}(\text{Socrates})$ $\text{Mortal}(\text{Socrates})$ = Socrates is mortal.

3. Difference from Rule:

- **Fact** = directly true statement.
- **Rule** = a conditional statement.
 - Example:
 - Fact: “ Socrates is a man.”
 - Rule: “ All men are mortal.”

From fact + rule $\rightarrow$ new fact: “ Socrates is mortal.”

There are different ways of representing knowledge

- **e.g.** Propositional Logic
 - predicate logic,
 - semantic networks,
 - extended semantic net,
 - frames,
 - conceptual dependency, etc...

What is Propositional Logic?

- **Propositional Logic (PL)** is the simplest form of logic used in AI.

- It deals with **statements (propositions)** that can only be **true (T)** or **false (F)**
- It doesn't break down into smaller parts like objects or relationships (that's handled by **Predicate Logic**).

Example of propositions:

- "It is raining." (True/False)
- "The light is on." (True/False)

Components of Propositional Logic

1. **Propositions (Atomic Statements)**
 - Represented by symbols like **P, Q, R, ...**
 - Example:
 - P: "It is raining."
 - Q: "I will take an umbrella."

To represent complex sentences or natural language sentences propositional logic is not sufficient.

Logical Connectives (Operators)

- Used to build complex sentences.

Symbol	Operator	Meaning	Example
$\neg$	NOT	Negation	$\neg P \rightarrow$ "It is not raining."
$\wedge$	AND	Conjunction	$P \wedge Q \rightarrow$ "It is raining and I will take an umbrella."
$\vee$	OR	Disjunction	$P \vee Q \rightarrow$ "It is raining or I will take an umbrella."
$\rightarrow$	IMPLIES	If...then	$P \rightarrow Q \rightarrow$ "If it is raining, then I will take an umbrella."
$\leftrightarrow$	BICONDITIONAL	If and only if	$P \leftrightarrow Q \rightarrow$ "It is raining if and only if I take an umbrella."

Predicate Logic

- Predicate logic, also known as **First-Order Logic (FOL)**, is a formal system used in **artificial intelligence, mathematics and logic** to represent relationships between objects and make inferences. It extends **Propositional Logic** by incorporating **quantifiers, predicates and variables**, making it more expressive.
- **Components of Predicate Logic**
 - 1.**Constants (Objects)**: Represent specific entities in the domain(physical). Example: *Alice*, *5*, *Paris*
 - 2.**Variables**: Represent unspecified elements in the domain. Example: *x*, *y*, *z*
 - 3.**Predicates (Relations/Properties)**: Functions that define a property or a relation between objects. Example: *Loves(Alice, Bob)* (Alice loves Bob). Here Loves is predicate.
 - 4.**Functions**: Define deterministic mappings from objects to objects. Example: *Father(John)* = *Robert* (John's father is Robert)
 - 5.**Logical Connectives**: $\neg$ (Negation), $\wedge$ (AND), $\vee$ (OR), $\rightarrow$ (Implication), $\leftrightarrow$ (Biconditional)
 - 6.**Quantifiers**: Quantifiers help us express general rules (universal) and existence of something (existential) in predicate logic.

1. Universal Quantifier ($\forall$): If the sentence contains "For all", "For Every", "Anything", "for Any" – expresses generality. We use universal quantifier.
Example: $\forall x \text{ Loves}(x, \text{Chocolate}) \rightarrow (\text{Everyone loves chocolate})$

2. Existential Quantifier ($\exists$): If sentence contains "There exists", "For Some", "For at least one", "There exists some" – expresses existence.
Example: $\exists x \text{ Loves}(x, \text{Pizza})$ (At least one person loves pizza)

Predicate Logic

- Examples of Predicate Logic Sentences

1. Basic Statements (Predicate logic uses **predicates** to describe **properties of objects**)

- "John is a student." $\rightarrow$ Student(John)
- "Mary is a doctor." $\rightarrow$ Doctor(Mary)

2. Relationships Between Objects

- "Alice loves Bob." $\rightarrow$ Loves(Alice, Bob)
- "Tom is taller than Jerry." $\rightarrow$ Taller(Tom, Jerry)
- "Paris is the capital of France." $\rightarrow$ Capital(Paris, France)

- 3. Universal Quantifier ($\forall$) – "For all" (Used to say something is true for **every object**.)

- "All humans are mortal." $\rightarrow \forall x (\text{Human}(x) \rightarrow \text{Mortal}(x))$
- "Every student studies." $\rightarrow \forall x (\text{Student}(x) \rightarrow \text{Studies}(x))$
- "If a person is a teacher, they teach." $\rightarrow \forall x (\text{Teacher}(x) \rightarrow \text{Teaches}(x))$

- 4. Existential Quantifier ($\exists$) – "There exists"

- "Some people like ice cream." $\rightarrow \exists x (\text{Person}(x) \wedge \text{Likes}(x, \text{IceCream}))$
- "There exists a person who is a millionaire." $\rightarrow \exists x (\text{Person}(x) \wedge \text{Millionaire}(x))$

- "Some dogs are friendly." $\rightarrow \exists x (\text{Dog}(x) \wedge \text{Friendly}(x))$

Semantic Nets (Semantic Networks)

A **Semantic Net** is a **graph-based** knowledge representation technique

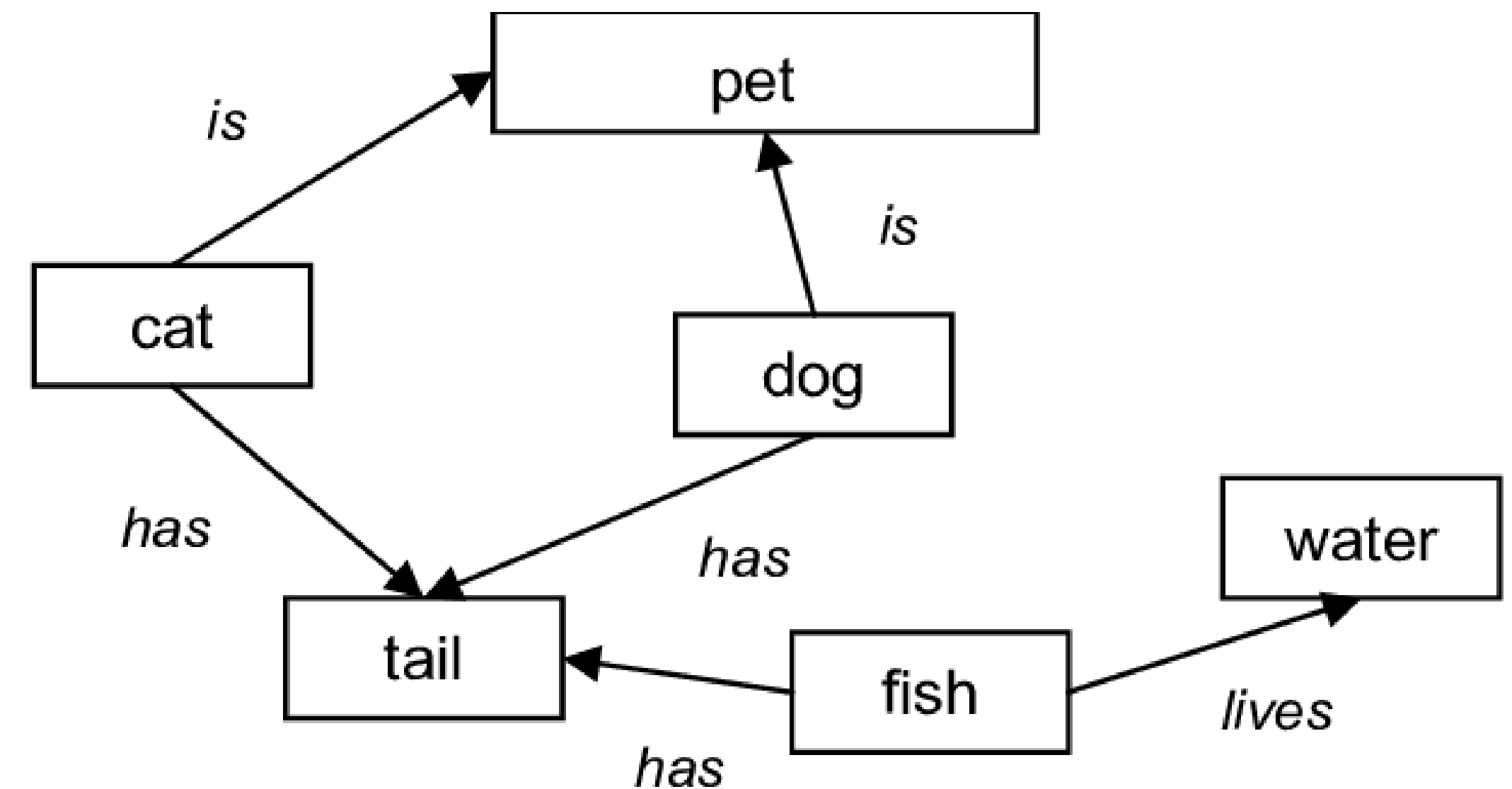
- where:
 - **Nodes** represent **concepts or objects**
 - **Edges (Links)** represent **relationships** between concepts
 - Labels on edges describe the type of relationship

- **Types of Links in Semantic Networks:**

- IS-A (class membership and inheritance)
- HAS-A (part-whole relationships)
- Property links (attributes)
- Action links (capabilities)

- **Example Semantic Net**

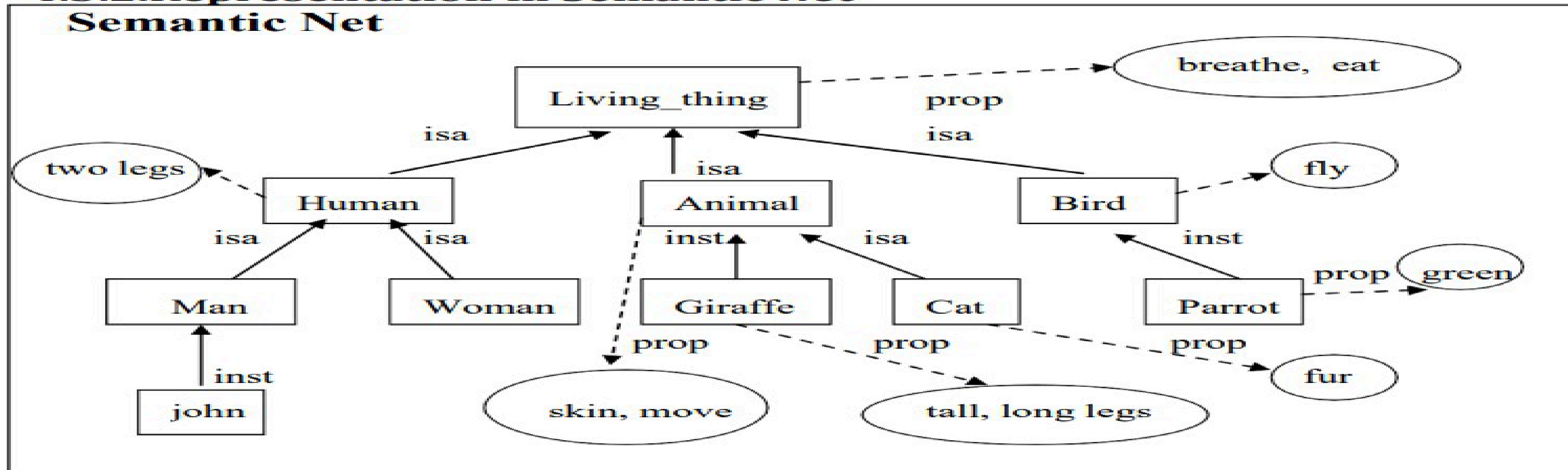
- A cat is a pet
- A dog is a pet
- A cat has a tail



- A dog has a tai

Semantic Nets (Semantic Networks)

4.3.2. Representation in Semantic Net Semantic Net



Using inheritance, the AI system can deduce new information:

- John is a Human → Humans are Living things → Living things breathe and eat → Therefore, John breathes and eats.
- Parrot is a Bird → Birds are Living things →

Therefore, Parrot can fly, breathe, and eat.

FRAMES

- Natural language understanding requires inference i.e., assumptions about what is typically true of the objects or situations under considerations, such information is coded into structures called Frames. To truly *understand* a sentence, a computer must go beyond words – it needs to make inferences (logical assumptions) about what is *usually true* in the real world.
- Example:
If the sentence says,
“John went to a restaurant and ordered food,”
the AI must infer that:
 - ▶ Restaurants serve food.
 - ▶ John is probably hungry.
 - ▶ Someone (like a waiter) will bring the food. These common-sense facts and typical situations are not directly written in the text, but the AI must know them.
- To store such typical, structured, reusable knowledge, AI uses Frames – special data structures that describe a situation, object, or concept along with its usual attributes and relationships.

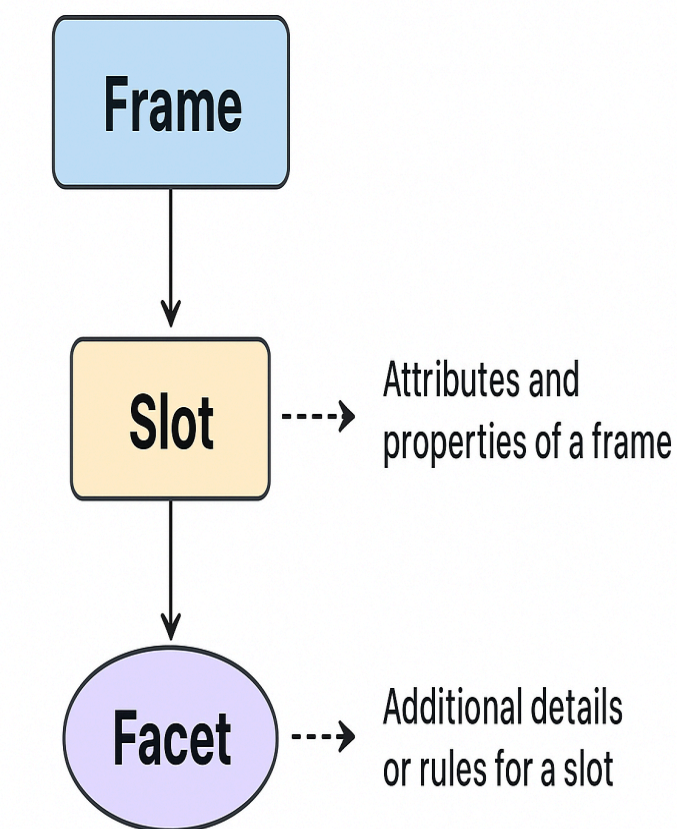
- A frame is record like data structures which consists of a collection of attributes and its value to describe an entity in the world.
- In this technique, the knowledge is stores via slots and fillers. Frame also called as Slots- Fillers Knowledge representation in AI.
- Components of Frame
 - Frame Name – Name of the frame
 - Slots - Slots are attributes or properties of a frame.
 - Fillers - values
- Each frame consists of a set of attributes and values, forming a template for

Book	
Slot	Filler
Name	Artificial Intelligence
Author	Peter Norvig
Year	2022
Price	Rs.750

understanding specific situations.

Facets:

- In AI (Artificial Intelligence), a Frame represents knowledge using slots and values. But sometimes, we need more details about each slot – like what kind of value it can take, what the default is, or what should happen when the value changes.
- These extra details are called Facets.
- Facets provide additional details or rules for slots, defining acceptable values or specifying how slots should be used.



- The various aspect of a slot is known as facts.
- Facets are features of frames which enable us to put rules on the frames
 - Value – Value of the slots

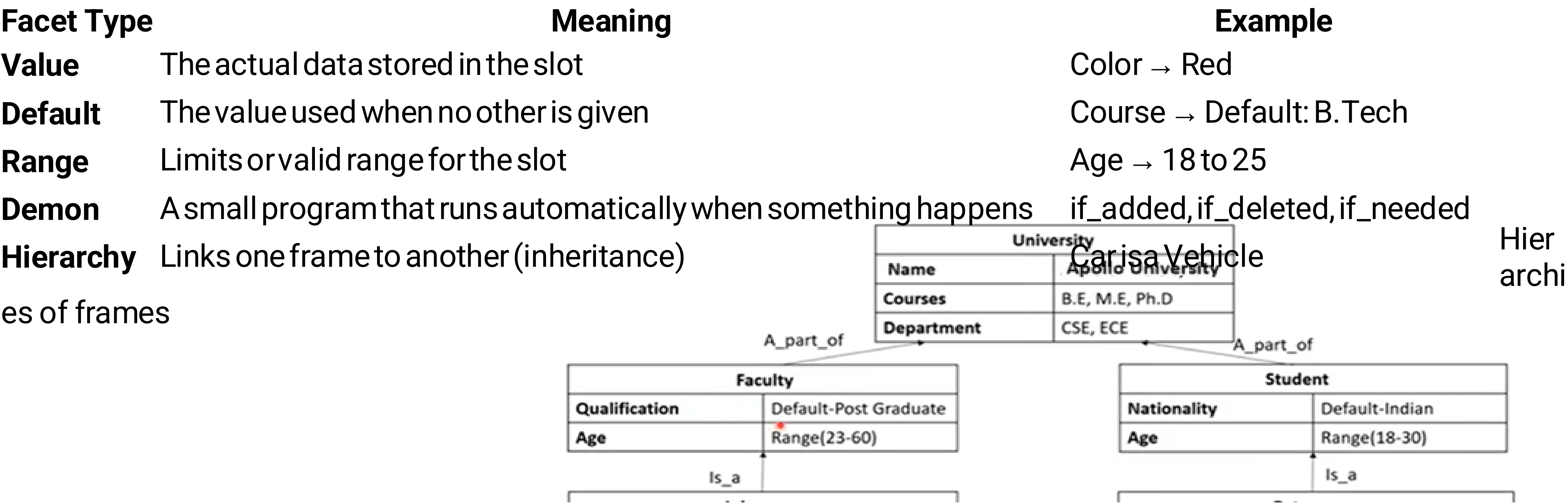
- Default – default value of the slots
- Range – indicates the range of integer
- Demons (Functions) – if_added, if_deleted, if_needed, etc..
- Frame: Student

Slot: Name, Value → "Priya"

Slot: Age, Facet: Range → 18–25

Slot: Course, Facet: Default → "B.Tech"

Slot: Marks Facet: if_added → "Update overall grade"



John's Qualification not given → use default → *Post Graduate*.
But since it's given as *Ph.D*, that overrides the default.

If Peter's Age is not entered → use facet rule → calculate using DOB.

Both John and Peter belong to the CSE department (inherited from University departments).

Inheritance

- Frame inheritance is a method used in knowledge representation systems to manage and organize information efficiently.
- It allows one frame (child) to inherit attributes and properties from another frame (parent), creating a hierarchical structure. This method facilitates the reuse and extension of existing knowledge.
- It facilitates inferencing (drawing conclusions) in semantic networks. Frame inheritance enables an AI system to think logically by passing down properties or facts through hierarchical relationships in a semantic network. This allows the system to infer or deduce new knowledge automatically from existing information.

- Frame: Living_Thing

Slot: CanBreathe → Yes

Slot: CanEat → Yes

Frame: Animal (isa Living_Thing)

Slot: CanMove → Yes

Frame: Bird (isa Animal)

Slot: HasFeathers → Yes

Now:

- Bird automatically inherits:
 - CanBreathe and CanEat from Living_Thing
 - CanMove from Animal

This reduces duplication and makes knowledge easy to maintain and extend.

Inheritance in frames is similar to inheritance in semantic networks.

- In a semantic net, concepts (nodes) are linked by "isa" and "instance-of" relationships.
- When you move upward through the network, properties are inherited from parent to child.

Living Thing

|

|--- Human

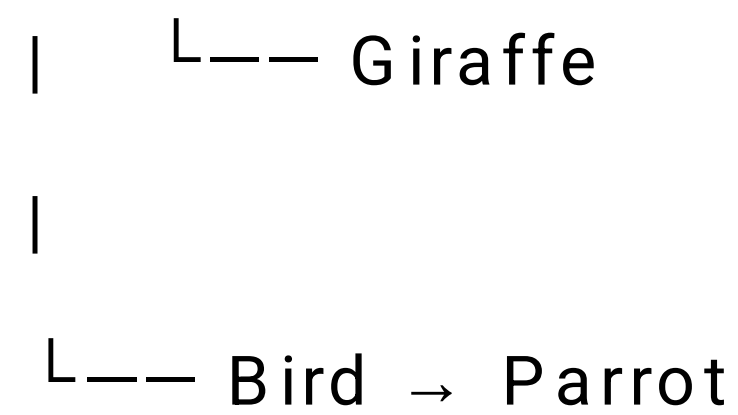
| |--- Man → John

| |--- Woman

|

|--- Animal

| |--- Cat



Key Concepts of Frame Inheritance

- 1. **Parent Frame:** The frame from which attributes and properties are inherited. It defines general attributes that are common to all its child frames.
- 3. **Child Frame:** The frame that inherits attributes and properties from the parent frame. It can add new attributes or override existing ones to represent more specific information.
- 5. **Inheritance Hierarchy:** A tree-like structure where frames are organized hierarchically. Each child frame can inherit from multiple parent frames, forming a network of relationships.
- 7. **Overriding:** When a child frame modifies or replaces an attribute inherited from the parent frame with a more specific value or definition.
- 9. **Extension:** Adding new attributes or properties to a child frame that are not present in the parent frame.

Frame Inheritance Works

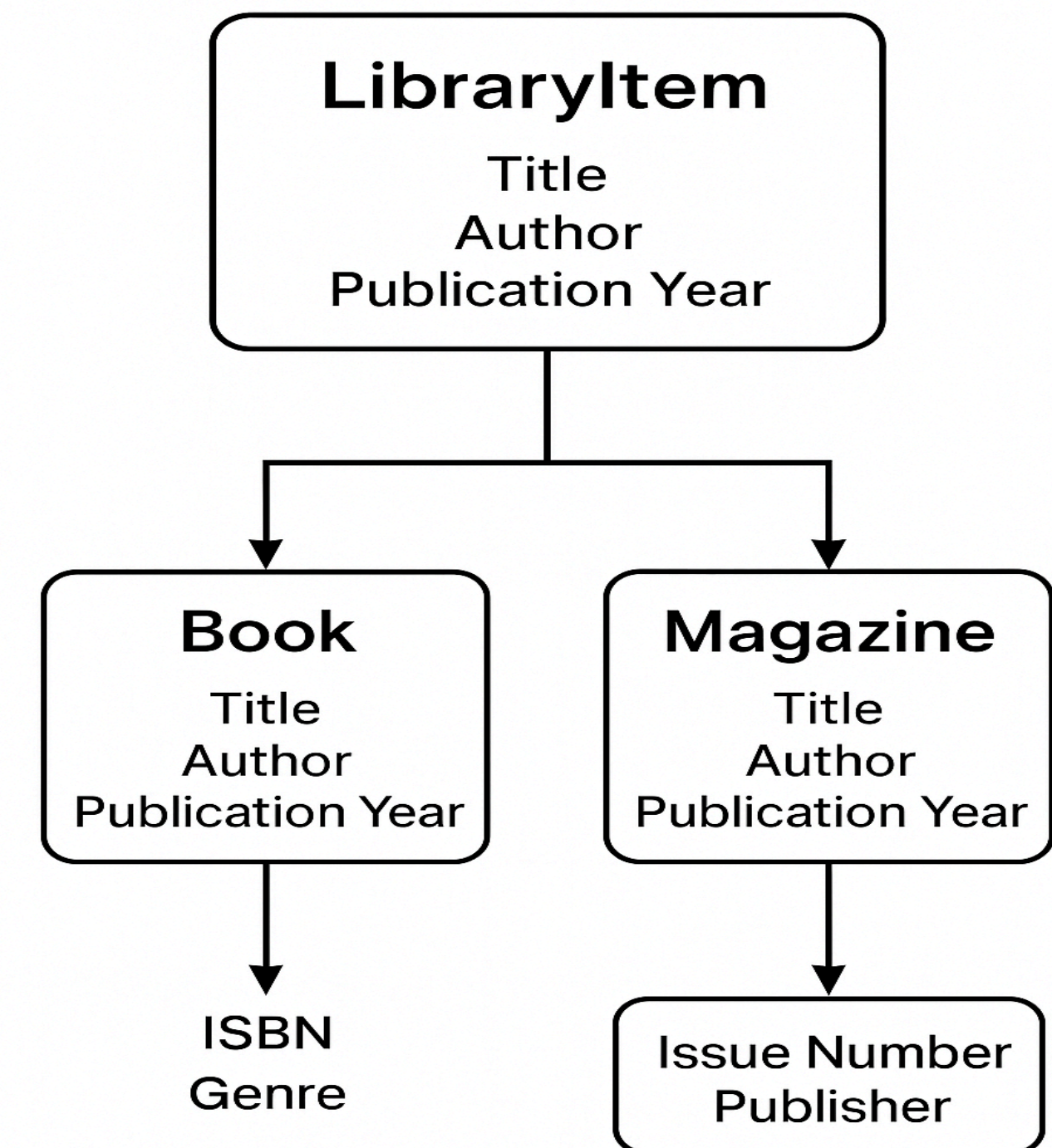
1. Define Parent Frame: Create a general frame with common attributes. For example, a "Vehicle" frame might include attributes like "Make," "Model," and "Year."

2. **Create Child Frame:** Define a more specific frame that inherits from the parent frame. For example, a "Car" frame might inherit attributes from the "Vehicle" frame and add specific attributes like "Number of Doors."
4. **Use Inherited Attributes:** The child frame automatically includes all attributes from the parent frame, providing a structured way to build on existing knowledge.
6. **Override or Extend:** Modify or add attributes in the child frame as needed to refine the representation. For example, the "Car" frame might override the "Year" attribute to specify a range of acceptable values.

Example of Frame Inheritance

Let's consider an example with a hierarchy of frames in a library system:

- **Parent Frame:** "Library Item"
 - **Attributes:**
 - **Title**
 - **Author**
 - **Publication Year**
- **Child Frame 1:** "Book" (inherits from "Library Item")
 - **Inherited Attributes:** Title, Author, Publication Year
 - **Extended Attributes:**
 - **ISBN**
 - **Genre**
- **Child Frame 2:** "Magazine" (inherits from "Library Item")
 - **Inherited Attributes:** Title, Author, Publication Year
 - **Extended Attributes:**
 - **Issue Number**
 - **Publisher**



Applications of Frames in AI

1. **Natural Language Processing (NLP)**: In NLP, frames are used to understand the context of words and sentences. For example, a "booking" frame might be used to interpret requests for reservations, extracting relevant information such as date, time, and number of people.
3. **Expert Systems**: Expert systems use frames to represent knowledge about specific domains. For instance, a medical diagnosis system might employ frames to represent various diseases, symptoms, and treatment options.
5. **Robotics**: Frames help robots make sense of their environment by providing structured information about objects and their properties. This allows robots to perform tasks such as object recognition and manipulation.
7. **Cognitive Modeling**: Frames are used in cognitive modeling to simulate human thought processes. By representing knowledge in frames, researchers can create models that mimic human reasoning and decision-making.

Advantages of Using Frames

- **Organized Knowledge**: Frames help in structuring information in a way that mirrors real-world scenarios, making it easier for AI systems to understand and process.
- **Flexibility**: Frames can be easily modified or extended to incorporate new information or adapt to changing contexts.
- **Reusability**: Once defined, frames can be reused across different applications or scenarios, promoting consistency and efficiency.

Constraint Propagation

- Artificial Intelligence (AI) encompasses a variety of methods and techniques to solve complex problems efficiently. One such technique is constraint propagation, which plays a crucial role in areas like scheduling, planning, and resource allocation.
- Constraint propagation is a fundamental concept in constraint satisfaction problems (CSPs).
- A CSP involves variables that must be assigned values from a given domain while satisfying a set of constraints. **Constraint Satisfaction Problem (CSP)** is a type of problem where:
 - ▶ You have some variables,
 - ▶ Each variable has a set of possible values (called its *domain*),
 - ▶ And you have rules (constraints) that tell which combinations of values are allowed.
 - ▶ The goal is to assign values to all the variables so that all the rules are satisfied.

Constraint propagation aims to simplify these problems by reducing the domains of variables, thereby making the search for solutions more efficient.
- Key Concepts
 - Variables: Elements that need to be assigned values.
 - Domains: Possible values that can be assigned to the variables.
 - Constraints: Rules that define permissible combinations of values for the variables.
- Example
 - Consider a simple CSP with two variables, X and Y, each with domains {1, 2, 3}, and a constraint $X \neq Y$.

Y. Constraint propagation will iteratively reduce the domains as follows:

- If X is assigned 1, then Y cannot be 1, so Y's domain becomes {2, 3}.
 - If Y is then assigned 2, X cannot be 2, so X's domain is reduced to {1, 3}.
- This process continues until a stable state is reached.

❖ Let's say we have two variables:

- $X = \{1, 2, 3\}$
- $Y = \{1, 2, 3\}$

and a rule (constraint):

$X \neq Y \rightarrow X \text{ and } Y \text{ cannot have the same value.}$

Now we must find values for X and Y that follow this rule.

Possible solutions:

(X=1, Y=2), (X=2, Y=3), (X=3, Y=1), etc.

Invalid: (X=2, Y=2) because they are equal.

Real-life examples:

- **Timetable scheduling** – no two classes at the same time in the same room.
- **Map coloring** – neighboring regions must have different colors

Constraint Propagation

- **Applications of Constraint Propagation**

Constraint Propagation is used in many AI problem-solving domains where several conditions (constraints) must be satisfied simultaneously. It helps by reducing the search space and making solutions faster and more efficient.

- **Scheduling**

- In scheduling problems, tasks must be assigned to time slots without conflicts. Constraint propagation helps by reducing the possible time slots for each task based on constraints like availability and dependencies.
- Example: In an exam schedule, two exams for the same student cannot overlap. Propagation removes invalid time slots early.

Planning

- AI planning involves creating a sequence of actions to achieve a goal. Constraint propagation simplifies the planning process by reducing the possible actions at each step, ensuring that the resulting plan satisfies all constraints.
- Example: In a robot navigation task, propagation removes moves that lead to blocked

paths or violate energy limits.

Resource Allocation

- In resource allocation problems, resources must be assigned to tasks in a way that meets all constraints, such as capacity limits and priority rules. Constraint propagation helps by narrowing down the possible assignments, making the search for an optimal allocation more efficient.

Applications of Constraint Propagation

Constraint Propagation is used in many AI problem-solving domains where several conditions (constraints) must be satisfied simultaneously. It helps by reducing the search space and making solutions faster and more efficient.

Scheduling



Goal: Assign tasks to time slots without conflicts

Use of Constraint Propagation:

It narrows down possible time slots for each task based on constraints like availability and dependencies

Planning

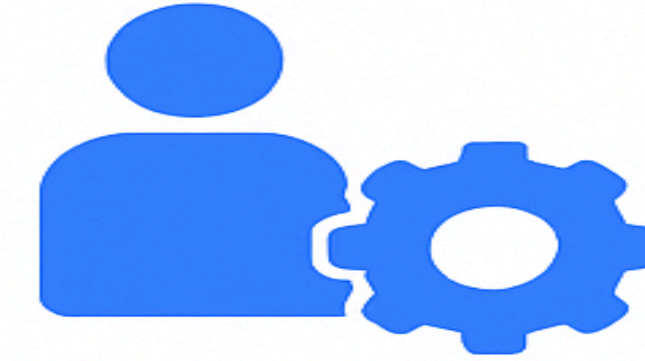


Goal: Generate a valid sequence of actions to reach a goal

Use of Constraint Propagation:

Reduces the number of possible actions at each step by enforcing constraints such as pre-conditions or effects

Resource Allocation



Goal: Assign available resources to tasks efficiently

Use of Constraint Propagation:

Helps eliminate impossible resource-task combinations based on capacity limits and priority or cost rules

Representing knowledge using rules

- Rule-based systems, a foundational technology in artificial intelligence (AI), have been instrumental in decision-making and problem-solving across various domains. They function using a set of predefined rules and logical reasoning to perform tasks, make decisions, and draw conclusions.
- Components of a Rule-Based System

- **Rules:** Rules are the main building blocks of a rule-based system. They tell the system *how to behave* or *what decision to make* when certain conditions are met. A rule generally follows the format "IF condition THEN action."

Example, in an expert system for medical diagnosis, a rule might be "IF patient has fever AND cough THEN consider flu."

Rules help the system make decisions in a logical, step-by-step way — just like a human expert would. By combining many such rules, the system can handle complex reasoning tasks.

Example 2 – Banking Expert System:

IF a customer's account balance is below ₹ 500 **AND** there are unpaid dues **THEN** send a low-balance alert and restrict further withdrawals.

Knowledge Base: This is the repository where all the rules and facts are stored. The knowledge base is built from domain-specific knowledge and can be manually derived from expert input. It acts as the

foundation for reasoning — the inference engine uses this stored knowledge to draw conclusions or make decisions.

It stores all the **rules** (the “ IF– THEN” statements) and **facts** (the known information or data) that the system uses to make decisions.

Inference Engine: The inference engine is the component that applies the rules to the knowledge base to derive conclusions or make decisions. It interprets the rules, processes them against the current facts or data, and determines the appropriate actions or outputs. **How it works:**

1. It reads the rules available in the system (from working memory).
2. It checks which rules in the knowledge base match those input.
3. It applies the matching rules to produce new conclusions or actions.

It allows the system to **reason logically**, just like a human expert — moving from known facts to new information or decisions.

- **Working Memory:** This is a dynamic component that holds the current facts or data being processed by the system. It is updated as the inference engine applies rules and new information becomes available.

- **User Interface:** In many rule-based systems, the user interface allows users to interact with the system, input data, and receive outputs or recommendations. The user interface ensures the system is **easy to use**, **interactive**, and **helpful**, even for users who are not experts in AI or programming.

Representing knowledge using rules

Rule-Based Systems Work:

- The working of a rule-based system happens in a series of logical steps. These steps allow the system to take input, apply rules, and produce meaningful results — just like how a human expert reasons.
- **Data Input:** The system receives input data from the user or another source. This data can range from simple numerical values to complex information like patient symptoms or transaction records.
- **Rule Matching:** The **inference engine** compares the input data with the **rules** stored in the **knowledge base**. It searches for rules where the **IF conditions** match the given facts. **Example:**

If the rule says “IF temperature > 38°C THEN fever,”
and the input data shows 39°C, this rule is matched.

Rule Execution: Once a rule is matched, the inference engine executes the corresponding action. This might involve updating the working memory, deriving new facts, or generating an output.

- **Conflict Resolution:** In cases where multiple rules are triggered simultaneously, the inference engine uses conflict resolution strategies to determine which rule to apply first. Common strategies include prioritizing rules based on specificity or order of entry. Sometimes, **more than one rule** may apply at the

same time.

- The inference engine uses **conflict resolution strategies** to decide which rule should be executed first.

Common strategies:

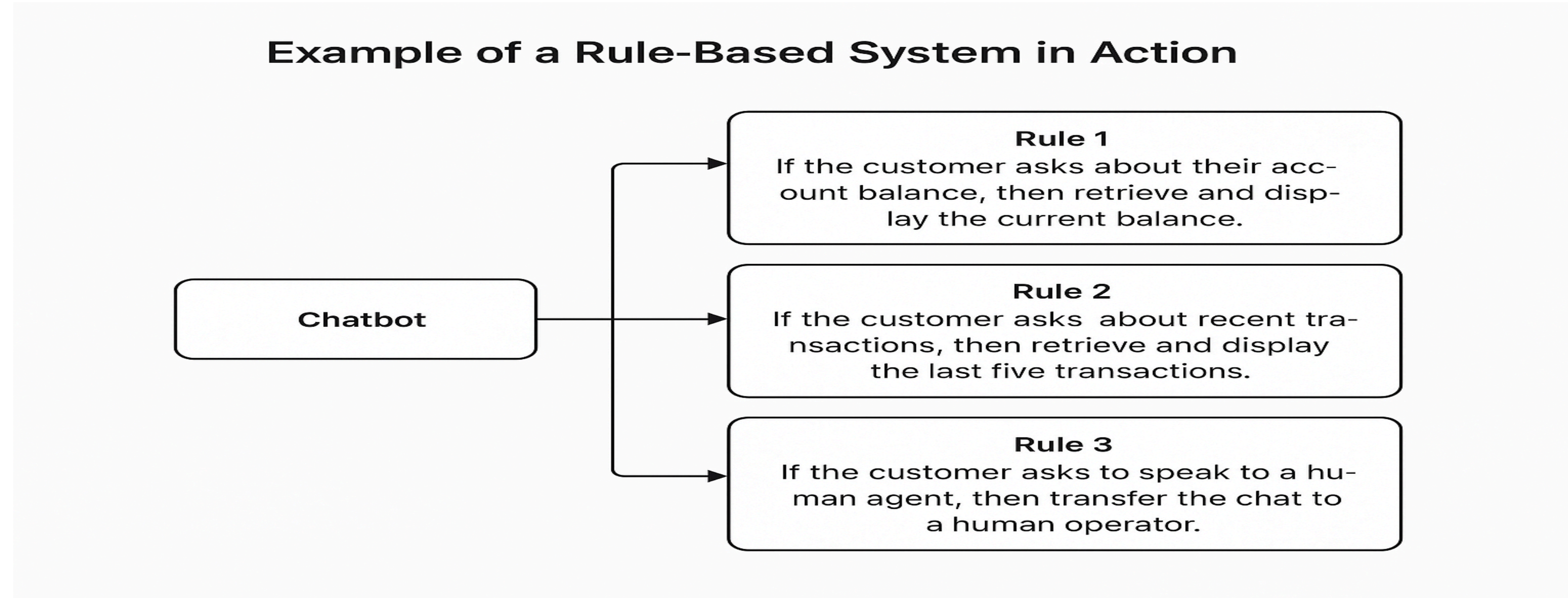
- **Specificity:** Prefer more specific rules over general ones.
 - **Priority order:** Follow the order in which rules were defined.
 - **Recency:** Give preference to rules based on recent facts.
-
- **Output Generation:** The system generates an output based on the executed rules. This output can be a decision, recommendation, or another form of response. For example, in a medical diagnosis system, the output might be a suggested treatment plan.

Representing knowledge using rules

Example of a Rule-Based System in Action

Consider a simplified example of a rule-based system used in a customer service chatbot:

- **Rule 1:** *If the customer asks about their account balance, then retrieve and display the current balance.*
- **Rule 2:** *If the customer asks about recent transactions, then retrieve and display the last five transactions.*
- **Rule 3:** *If the customer asks to speak to a human agent, then transfer the chat to a human operator.*



2. Healthcare: Medical Diagnosis System

A rule-based system can assist doctors in diagnosing diseases based on patient symptoms.

Scenario: Flu Diagnosis

- **Input:** Patient symptoms – fever, cough, and sore throat.
- **Knowledge Base (Rules):**
 - Rule 1: IF fever > 100° F AND cough THEN suggest “ flu diagnosis.”
 - Rule 2: IF fever > 102° F AND sore throat THEN recommend “ consult a doctor.”
 - Rule 3: IF fever < 100° F AND no cough THEN suggest “ rest and hydration.”

Process:

1. The patient enters their symptoms into the system.
2. The **Inference Engine** matches these symptoms with the rules in the **Knowledge Base**.
3. Based on the symptoms “ fever > 100° F and cough,” Rule 1 is triggered.
4. **Output:** The system recommends “ flu diagnosis.”

Result:

The user receives advice based on predefined medical rules, improving efficiency and reducing diagnostic errors.

Rule-Based Deduction Systems

Definition:

A **Rule-Based Deduction System** is a type of Artificial Intelligence (AI) system that **draws logical conclusions** from a set of given facts and rules.

It uses reasoning — specifically **deductive reasoning** — to infer new knowledge from existing information. **Deductive reasoning** is a type of logical thinking where we start from general rules or facts and use them to arrive at a specific conclusion that must be true.

Example 1

Rule (General statement):

All birds have feathers.

Fact (Specific case):

A parrot is a bird.

Conclusion (What must be true):

Therefore, a parrot has feathers.

This is **deductive reasoning** — we started from a general truth (“ All birds have feathers”) and logically deduced a specific fact (“ Parrot has feathers”).

How It Works:

1. Knowledge Representation:

- The system stores knowledge in the form of rules and facts.
- Each rule follows the format:
IF condition THEN conclusion

2. Deductive Reasoning:

- The system applies logical rules to known facts to derive new facts.
- This process continues until no more conclusions can be drawn.
- It mimics how humans' reason logically from known truths.

3. Inference Process:

- The **inference engine** is used to apply the rules and perform deductions.
- It can use **forward chaining** (from facts to conclusions) or **backward chaining** (from goal to facts).

Example:

Rules:

1. IF it rains THEN the ground gets wet.
2. IF the ground gets wet THEN the grass becomes slippery.

Fact:

It rains.

Deduction:

➡ From Rule 1 → The ground gets wet.

➡ From Rule 2 → The grass becomes slippery.

So, the system deduces:

The grass becomes slippery (new fact derived logically).

Types of Rule-Based Systems

Rule-based systems can be categorized based on their reasoning approach. The two primary types are **Forward Chaining** and **Backward Chaining** systems. These approaches determine how the system processes rules and input data to reach a conclusion.

1. Forward Chaining Systems

Forward Chaining follows a **data-driven** approach. It starts with known facts or input data and applies rules step by step to reach a conclusion.

How It Works:

1. The system begins with the available input data.
2. It checks the rules in the **Knowledge Base** to identify which rules match the input.
3. Rules are executed sequentially until a conclusion is reached.

Example: Medical Diagnosis System

- **Input:** Patient reports fever $> 100^{\circ}$ F and headache.
 - **Rules:**
 - Rule 1: IF fever $> 100^{\circ}$ F AND headache THEN suggest “ flu.”
 - Rule 2: IF flu AND sore throat THEN recommend “ flu medication.”
 - **Process:** The system starts with the symptoms (data), matches Rule 1, and proceeds step by step to additional rules.
 - **Output:** Suggest “ flu” and recommend medication.
- Use Case:** Forward chaining is commonly used in **expert systems** for medical diagnosis and troubleshooting problems.

2. Backward Chaining Systems

Backward Chaining follows a **goal-driven** approach. It starts with a desired goal and works backward to identify the conditions or rules that satisfy that goal.

How It Works:

1. The system begins with a hypothesis or goal (e.g., “ flu diagnosis”).

2. It looks for rules that can prove the hypothesis.
3. It works backward through the conditions of those rules until it finds supporting facts.

Example: Debugging System

- **Goal:** Determine if a system error is caused by insufficient memory.
- **Rules:**
 - Rule 1: IF system crash AND memory < 2GB THEN cause is “ insufficient memory.”
 - Rule 2: IF process usage > 90% THEN memory < 2GB.
- **Process:** The system starts with the goal “ insufficient memory” and works backward to verify the conditions step by step.
- **Output:** Confirms that memory is insufficient.

Use Case: Backward chaining is widely used in **logical proofs** and **debugging systems** where the goal is predefined.

Example:

Goal: Is the grass slippery?

→ Check if the ground is wet.

→ Check if it rained.

If both are true → conclude “ Grass is slippery.”

Difference between

Representing Knowledge Using Rules vs Rule-Based Deduction Systems

Concept

Representing Knowledge Using Rules

Meaning

It is a method of storing or expressing knowledge in the form of “ IF– THEN” rules.

Focus

How knowledge is written and represented.

Nature

Static – just expresses information.

Example:

1. IF it rains THEN the ground gets wet.
2. IF the ground gets wet THEN the grass becomes slippery.

These are just **rules stored in the knowledge base**.

No deduction has been done yet.

We only *store* the rules – no reasoning happens yet.

Rule-Based Deduction Systems

It is a complete AI system that uses those rules to perform logical reasoning and draw conclusions.

How the system *uses* that knowledge to think and deduce.

Dynamic – applies logic and reasoning to infer new information.

Example

Representing Knowledge Using Rules Rule-Based Deduction System

Now, the system uses these rules to reason and draw conclusions.

Given the fact:

➡ *It rains.*

The system applies rules:

From Rule 1 → The ground gets wet.

From Rule 2 → The grass becomes slippery.

New fact deduced: The grass becomes slippery.

So here, deductive reasoning is performed automatically by the inference engine.

Reasoning under uncertainty

An agent working in real environment almost never has access to whole truth about the environment. Therefore, agent need to work under uncertainty. With knowledge representation  we might write $A \rightarrow B$, which means if A is true then B is True, but consider a situation where we are not sure about whether A is true or not then we cannot express these statement. This situation is called Uncertainty.

Uncertainty in Artificial Intelligence (AI) refers to the inability of models to make fully confident predictions due to incomplete, ambiguous, or noisy data. AI systems must account for uncertainty to make accurate and reliable decisions, especially in dynamic environments where information is inconsistent or evolving.

Why Uncertainty Arises in AI

Uncertainty in AI can occur due to:

Incomplete data – not all facts are available.

Imagine an AI system designed to diagnose diseases based on patient symptoms.

- Suppose a patient visits a clinic with fever and fatigue, but the AI does not know about other symptoms like rash or cough because the patient did not report them.
- Due to this incomplete data, the AI may miss a possible disease, like measles, which requires knowledge of the rash.

This shows that not having all relevant facts creates uncertainty in AI' s reasoning.

Noisy data – sensor readings or inputs may contain errors.

Ambiguity – multiple interpretations of the same data.

Examples of Uncertainty in Real-World AI Applications:

- **Autonomous Vehicles:** AI-powered self-driving cars must navigate unpredictable road conditions, such as sudden pedestrian movement or bad weather.

Scenario: A self-driving car is driving on a rainy highway. Suddenly, a cyclist swerves into the lane to avoid a pothole.

Uncertainty: The AI must quickly predict the cyclist's path and decide whether to brake, swerve, or continue. The unpredictability of human behaviour and weather conditions introduces uncertainty in decision-making.

- **Healthcare Diagnostics:** AI models analysing medical images face uncertainty due to variability in symptoms, leading to multiple possible diagnoses.

Example:

Scenario: An AI model analyzing chest X-rays must detect early signs of pneumonia. Some patients may have overlapping symptoms with COVID-19.

Uncertainty: The variability in how diseases appear in different patients can lead the AI to suggest multiple possible diagnoses, making it difficult to determine the correct one without further tests.

- **Natural Language Processing (NLP):** AI chatbots and language models encounter contextual ambiguity, making it difficult to infer user intent accurately. **Example:**

Scenario: A customer types in a support chatbot: “ My account isn’ t working.”

Uncertainty: The AI must infer whether the issue is a forgotten password, a frozen account, or a technical system error. The same sentence can have multiple meanings depending on context, causing ambiguity in response generation.

- **Fraud Detection:** In finance, AI models must distinguish between genuine and fraudulent transactions, often dealing with uncertain patterns in data.

Example:

- **Scenario:** A bank’ s AI monitors transactions and sees a customer suddenly making a large purchase in a foreign country.
- **Uncertainty:** It’ s unclear whether this is a legitimate transaction or a fraudulent activity. The AI must weigh patterns from past behaviour, transaction location, and timing, all of which may be ambiguous or incomplete.

To deal with Uncertainty we have three possible solutions.

They are:

1.Review of probability.

2.Baye' s Probabilistic Interferences

3.Demster-shafer Theory

- **1. Probability:** It is a way to measure how likely something is to happen, typically expressed as a number between 0 and 1. In AI, we use probabilities to understand and make predictions when the information we have is uncertain or incomplete.

Sample Space (S): Set of all possible outcomes.

Example: For a dice roll, $S = \{1, 2, 3, 4, 5, 6\}$

Event (E): A subset of the sample space.

Example: Getting an even number $E = \{2, 4, 6\}$

Probability of Event (P(E)):

$$P(E) = \frac{\text{Number of favorable outcomes}}{\text{Total number of outcomes}}$$

where $0 \leq P(E) \leq 1$

Rules of Probability

Addition Rule: $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

Multiplication Rule: $P(A \cap B) = P(A) \times P(B|A)$

Complement Rule: $P(\text{not } A) = 1 - P(A)$

Example

If the probability of rain today is 0.7, then the probability of *no rain* is $1 - 0.7 = 0.3$

(b) Addition rule (general)

$$P(A \cup B) = P(A) + P(B) - P(A \cap B).$$

Why: Adding $P(A)$ and $P(B)$ counts the overlap $P(A \cap B)$ twice, so subtract it once.

Example (dice):

Let $A = \{\text{even}\} = \{2, 4, 6\}$, $B = \{>3\} = \{4, 5, 6\}$.

Then $A \cup B = \{2, 4, 5, 6\}$, $A \cap B = \{4, 6\}$.

$$P(A) = \frac{3}{6}, P(B) = \frac{3}{6}, P(A \cap B) = \frac{2}{6}$$

So

$$P(A \cup B) = \frac{3}{6} + \frac{3}{6} - \frac{2}{6} = \frac{4}{6}.$$

$P(A)$: Probability that event **A** occurs.

$P(B)$: Probability that event **B** occurs.

$P(A \cap B)$: Probability that **both A and B** happen together.

$P(B | A)$: Probability that **B occurs given A has already happened**.

$P(A | B)$: Probability that **A occurs given B has already happened**.

(c) Multiplication rule / Conditional probability

$$P(A \cap B) = P(A) P(B | A) = P(B) P(A | B).$$

Definition of conditional probability:

$$P(B | A) = \frac{P(A \cap B)}{P(A)} \quad (\text{provided } P(A) > 0).$$

Intuition: $P(B | A)$ is the probability that B happens *given* that A has happened.

Example (cards): Draw one card from a standard 52-card deck.

Let $A = \{\text{card is a heart}\}$, $B = \{\text{card is an ace}\}$.

$$P(A) = \frac{13}{52} = \frac{1}{4}, P(B) = \frac{4}{52} = \frac{1}{13}, P(A \cap B) = P(\text{ace of hearts}) = \frac{1}{52}.$$

Then

$$P(B | A) = \frac{1/52}{1/4} = \frac{1}{13},$$

What is Probability?

- Probability expresses **how likely an event is to occur**.
- It is always a number **between 0 and 1**.
 - **0** → **impossible event**
 - **1** → **certain event**

Examples:

- Tossing a coin:
 $P(\text{Head})=0.5$, $P(\text{Tail})=0.5$
- Rolling a dice:
 $P(\text{Even})=3/6=0.5$

Why Probability in AI?

AI systems often deal with **uncertainty** — incomplete, noisy, or ambiguous information.

Probability helps measure **how much uncertainty** is present in an event or belief.

Example:

“ There is an 80% chance (probability 0.8) that the patient has a cavity if they have a toothache.”

But not all toothaches are caused by cavities — the missing **20%** accounts for other unknown causes (like gum problems, sensitivity, etc.).

This uncertainty may arise from **ignorance** or **lack of complete knowledge**.

Probability summarizes our **degree of belief**, not the truth itself.

- The **sentence** (e.g., “ the patient has a cavity”) is either **true or false**.
- The **probability** (e.g., 0.8) shows **how strongly we believe** it’ s true, given current evidence.

a. Probability Statements and Evidence

A probability statement like

“ The probability that the patient has a cavity is 0.8” reflects the **agent’ s belief**, not the actual world state.

When new **evidence (percepts)** arrives, such as:

“ X-ray shows no decay,”

the probability is **updated** to reflect new information.

So probability in AI is **dynamic** — beliefs are **revised** as evidence changes.

b. Random Variables

“ A random variable is a part of the world we are uncertain about — like weather, coin toss, or test result.”

Types:

1. **Boolean Variable** → True / False

Example:

Cavity = True or Cavity = False

2. **Discrete Variable** → Fixed number of possible values

Example:

Weather = {Sunny, Rainy, Cloudy, Snow}

3. **Continuous Variable** → Any real value in a range

Example:

Temperature = 31.5° C

Example Equation:

For a coin:

$$P(H)=0.5 \Rightarrow P(T)=1 - 0.5=0.5$$

Atomic Events

“An atomic event is one complete situation of the world.”

Example:

If the world has 2 variables:

- Weather = {Sunny, Rainy}
- Temperature = {Hot, Cold}

Then possible **atomic events** are:

1. (Sunny, Hot)
2. (Sunny, Cold)
3. (Rainy, Hot)
4. (Rainy, Cold)

Properties:

- **Mutually Exclusive:** Only one can happen at a time.
Sum of all probabilities = 1

Bayes' probabilistic inferences

- Bayes Theorem in AI is perhaps the most fundamental basis for probability and statistics, more popularly known as Bayes' rule or Bayes' law. It allows us to revise our assumptions or the probability that an event will occur, given new information or evidence.
- Bayes' Theorem in AI
 - In probability theory, Bayes' theorem talks about the relation of the conditional probability of two random events and their marginal probability. In short, it provides a way to calculate the value of $P(B|A)$ by using the knowledge of $P(A|B)$.
 - Bayes' theorem is the name given to the formula used to calculate conditional probability.
 - The formula is as follows:
$$P(A|B) = (P(A) * P(B|A)) / P(B)$$

where,
 - $P(A)$ is the probability that event A occurs.
 - $P(B)$ defines the probability that event B occurs.
 - $P(A|B)$ is the probability of the occurrence of event A given that event B has already occurred.
 - $P(B|A)$ can now be read as: Probability of event B occurring given that event A occurred.

- The Bayes' Theorem is a basic concept in probability and statistics. It gives a model of updating beliefs or probabilities when the new evidence is presented. This theorem was named after Reverend Thomas Bayes and has been applied in many fields, ranging from artificial intelligence and machine learning to data analysis.

The Bayes' Theorem encompasses four major elements:

- **Prior Probability $P(A)$:** The probability or belief in an event A prior to considering any additional evidence, it represents what we know or believe about A based on previous knowledge.
- **Likelihood $P(B|A)$:** the probability of evidence B given the occurrence of event A. It determines how strongly the evidence points toward the event.
- **Evidence $P(B)$:** Evidence is the probability of observing evidence B regardless of whether A is true.
- **Posterior Probability $P(A|B)$:** The posterior probability is a revised belief regarding event A, informed by some new evidence B. It answers the question, "What is the probability that A is true given evidence B observed?"
- Bayes' theorem is highly relevant in AI due to its ability to handle uncertainty and make decisions based on probabilities.
- **Probabilistic Reasoning:** In many real-world scenarios, AI systems must reason under

uncertainty. Bayes' theorem allows AI systems to update their beliefs based on new evidence. This is essential for applications like autonomous vehicles, where the environment is constantly changing and sensors provide noisy information.

- **Machine Learning:** Bayes' theorem serves as the foundation for Bayesian machine learning approaches. These methods allow AI models to incorporate prior knowledge and update their beliefs as they see more data. This is particularly useful in scenarios with limited data or when dealing with complex relationships between variables.
- **Classification and Prediction:** In classification tasks, such as spam email detection or medical diagnosis, Bayes' theorem can be used to calculate the probability that a given input belongs to a particular class. This allows AI systems to make more informed decisions based on the available evidence.
- **Anomaly Detection:** Bayes' theorem is used in anomaly detection, where AI systems identify unusual patterns in data. By modeling the normal behavior of a system, Bayes' theorem can help detect deviations from this norm, signaling potential anomalies or security threats.

Dempster-Shafer theory

- Dempster Shafer Theory (DST) is an evidence theory, it combines all possible outcomes of the problem. Hence it is used to solve problems where there may be a chance that a piece of different evidence will lead to some different result. Mathematical framework for reasoning under uncertainty and incomplete information. It was proposed by Arthur Dempster in 1967. Developed further by Glen Shafer in 1976.

This deals with situation where evidence is uncertain and incomplete. It allows combining evidence from different sources. Support decision making by assigning degree of Belief

Key Concepts of Dempster-Shafer Theory

DST operates based on a few key concepts that help us manage uncertainty and conflicting information.

Let's see them:

1. Frame of Discernment: A frame of discernment is simply the set of all possible outcomes in a given situation.

2. Basic Probability Assignment (BPA) or Mass Function: It assigns belief to subsets of the frame of discernment, not individual elements. BPAs express the degree of confidence in a set of possibilities, given the available evidence. Importantly, the total of all BPAs in the frame must sum to 1.

3. Belief, Plausibility and Doubt:

- **Belief:** Represents the confidence in a hypothesis or situation based on the evidence.
- **Plausibility:** Measures how likely a hypothesis might still be true even if the evidence isn't fully conclusive.
- **Doubt:** Reflects uncertainty about a hypothesis showing how likely it is that the hypothesis is false.

Dempster-Shafer theory

Example

Let us consider a room where four people are present, A, B, C, and D. Suddenly the lights go out and when the lights come back, B has been stabbed in the back by a knife, leading to his death. No one came into the room and no one left the room. We know that B has not committed suicide. Now we have to find out who the murderer is.

To solve these there are the following possibilities:

- Either {A} or {C} or {D} has killed him.
- Either {A, C} or {C, D} or {A, D} have killed him.
- Or the three of them have killed him i.e.; {A, C, D}
- None of them have killed him {o}.

Power Set(Frame of discernment) and Mass Function

The Power Set of P consists of all possible subsets of P including the empty set and the full set.

For $P = \{A, C, D\}$, the power set $2P$ is: $2P = \{\emptyset, \{A\}, \{C\}, \{D\}, \{A, C\}, \{A, D\}, \{C, D\}, \{A, C, D\}\} = \text{Total } 2^3 = 8 \text{ subsets.}$

Given a power set, we assign **mass functions** to these subsets. The mass function $m(K)$ refers to the belief assigned to a particular subset, representing evidence that supports a particular hypothesis or combination of hypotheses.

Mass Function (m)

Definition:

- $m : 2^\Theta \rightarrow [0, 1]$
- $m(\emptyset) = 0$, and
- $\sum m(A_i) = 1$

Interpretation:

- Represents a belief assigned exactly to each subset

The **mass function** represents **how much belief (or support)** we assign **exactly** to each subset (not to its supersets or subsets).

Evidence

Witness 1

“A or C might be involved”

- $m(\{A\}) = 0.2$
 - $m(\{C\}) = 0.2$
 - $m(\{A, C\}) = 0.3$
 - $m(\Theta) = 0.3$ (uncertainty)
- Total: $0.2 + 0.2 + 0.3 + 0.3 = 1$

Witness 2

“C or D were near the body”

- $m(\{C\}) = 0.3$
 - $m(\{D\}) = 0.2$
 - $m(\{C, D\}) = 0.3$
 - $m(\Theta) = 0.2$
- Total: $0.3 + 0.2 + 0.3 + 0.2 = 1$

Next, we are going to make combination table based on the evidences

Make a Combination Table

Set	m1(Set)
{A}	0.2
{C}	0.2
{A, C}	0.3
{A, C, D}	0.3

Set	m2(Set)
{C}	0.3
{D}	0.2
{C, D}	0.3
{A, C, D}	0.2

m1(X)	m2(Y)	$X \cap Y$	$m1 \times m2$
{A}	{C}	$\emptyset$	$0.2 \times 0.3 = 0.06$
{A}	{D}	$\emptyset$	$0.2 \times 0.2 = 0.04$
{A}	{C,D}	$\emptyset$	$0.2 \times 0.3 = 0.06$
{A}	{A,C,D}	{A}	$0.2 \times 0.2 = 0.04$ ✓
{C}	{C}	{C}	$0.2 \times 0.3 = 0.06$ ✓
{C}	{D}	$\emptyset$	$0.2 \times 0.2 = 0.04$
{C}	{C,D}	{C}	$0.2 \times 0.3 = 0.06$ ✓
{C}	{A,C,D}	{C}	$0.2 \times 0.2 = 0.04$ ✓
{A,C}	{C}	{C}	$0.3 \times 0.3 = 0.09$ ✓
{A,C}	{D}	$\emptyset$	$0.3 \times 0.2 = 0.06$
{A,C}	{C,D}	{C}	$0.3 \times 0.3 = 0.09$ ✓
{A,C}	{A,C,D}	{A,C}	$0.3 \times 0.2 = 0.06$ ✓
{A,C,D}	{C}	{C}	$0.3 \times 0.3 = 0.09$ ✓
{A,C,D}	{D}	{D}	$0.3 \times 0.2 = 0.06$ ✓
{A,C,D}	{C,D}	{C,D}	$0.3 \times 0.3 = 0.09$ ✓
{A,C,D}	{A,C,D}	{A,C,D}	$0.3 \times 0.2 = 0.06$ ✓

To calculate K Value. All the intersections that are **empty** ($\emptyset$) represent **conflict** between the two sources of evidence.
Sum of all conflict masses = **K** (used in normalization).

Separate Conflict			
m1(X)	m2(Y)	$X \cap Y$	$m1 \times m2$
{A}	{C}	$\emptyset$	$0.2 \times 0.3 = 0.06$
{A}	{D}	$\emptyset$	$0.2 \times 0.2 = 0.04$
{A}	{C,D}	$\emptyset$	$0.2 \times 0.3 = 0.06$
{A}	{A,C,D}	{A}	$0.2 \times 0.2 = 0.04$ ✓
{C}	{C}	{C}	$0.2 \times 0.3 = 0.06$ ✓
{C}	{D}	$\emptyset$	$0.2 \times 0.2 = 0.04$
{C}	{C,D}	{C}	$0.2 \times 0.3 = 0.06$ ✓
{C}	{A,C,D}	{C}	$0.2 \times 0.2 = 0.04$ ✓
{A,C}	{C}	{C}	$0.3 \times 0.3 = 0.09$ ✓
{A,C}	{D}	$\emptyset$	$0.3 \times 0.2 = 0.06$
{A,C}	{C,D}	{C}	$0.3 \times 0.3 = 0.09$ ✓
{A,C}	{A,C,D}	{A,C}	$0.3 \times 0.2 = 0.06$ ✓
{A,C,D}	{C}	{C}	$0.3 \times 0.3 = 0.09$ ✓
{A,C,D}	{D}	{D}	$0.3 \times 0.2 = 0.06$ ✓
{A,C,D}	{C,D}	{C,D}	$0.3 \times 0.3 = 0.09$ ✓
{A,C,D}	{A,C,D}	{A,C,D}	$0.3 \times 0.2 = 0.06$ ✓

Conflict
(where $X \cap Y = \emptyset$):

= 0.06 + 0.04 + 0.06
+ 0.04 + 0.06 = 0.26

K=0.26

Now calculate complete mass(A)

Dempster's Rule of Combination

Combine two mass functions m_1 and m_2 :

$$m(A) = \frac{1}{1-K} \sum_{X \cap Y = A} m_1(X) \cdot m_2(Y)$$

Where $K = \sum_{X \cap Y = \emptyset} m_1(X) \cdot m_2(Y)$ (Conflict)

Valid Masses ($X \cap Y \neq \emptyset$)

Intersection	Total Mass
{A}	0.04
{C}	$0.06 + 0.06 + 0.04 + 0.09 + 0.09 = 0.34$
{A,C}	0.06
{D}	0.06
{C,D}	0.09
{A,C,D}	0.06

Total valid = 0.04 + 0.34 + 0.06 + 0.06 + 0.09 + 0.06 = 0.65

When we created the **Combination Table** earlier, we found pairs of sets (X,Y) from m_1 and m_2 . Each pair produced:

- an intersection $X \cap Y$
- a product $m_1(X) \times m_2(Y)$

Some intersections were **empty** ($\emptyset$) $\rightarrow$ **conflict**, so we ignore those.
The rest (non-empty intersections) are **valid masses** – they represent **consistent evidence** between m1 and m2.

Normalize

- To remove the conflict, normalize all non-conflict values by $1-K=1-0.26=0.74$
- Now divide each valid mass by 0.74:

Intersection	Final Mass = $m(X) / (1-K)$
{A}	$0.04 / 0.74 \approx 0.054$
{C}	$0.34 / 0.74 \approx 0.459$
{A,C}	$0.06 / 0.74 \approx 0.081$
{D}	$0.06 / 0.74 \approx 0.081$
{C,D}	$0.09 / 0.74 \approx 0.122$
{A,C,D}	$0.06 / 0.74 \approx 0.081$

Final Combined Mass Function

Hypothesis	Final Combined Mass
{C}	0.459 ☒ Most Believable
{A}	0.054
{A,C}	0.081
{D}	0.081
{C,D}	0.122
{A,C,D}	0.081

Dempster-Shafer theory

Advantages of Dempster Shafer Theory:

1. Medical Diagnosis: It helps combine multiple test results or symptoms to make a more accurate diagnosis by handling uncertainty in medical data.
3. Sensor Fusion: It integrates data from different sensors to create a cohesive and accurate understanding even when individual sensors provide conflicting information.
5. Machine Learning: It improves classification tasks by managing uncertainty in data allowing for more flexible and reliable predictions.
7. Robotics: It is used in robotics to combine uncertain data from various sources, helping robots make better decisions in dynamic environments.

Disadvantages of Dempster Shafer Theory:

- In this, computation effort is high, as we have to deal with 2^n sets